

Departamento de Eletrónica, Telecomunicações e
Informática

**LECTURE 11 SEQUENCE MODELS
&
TIME SERIES FORECASTING**

Petia Georgieva
(petia@ua.pt)

Outline

1. Sequence models – motivation
2. Recurrent Neural Networks (RNN)
3. Backpropagation through time
4. Long-Short Term Memory (LSTM)
5. Time Series Forecasting

Examples of sequence data

x (input)

y (output)

Speech recognition



==> “The quick brown fox jumped over the lazy dog.”

Sentiment classification

“There is nothing to like in this movie.”



DNA sequence analysis

AGCCCCTGTGAGGAACTAG

==> AG**CCCCTGTGAGGAACTAG**

Machine translation

Voulez-vous chanter avec moi?

==> Do you want to sing with me?

Video activity recognition



==> Running

Name entity recognition

Yesterday, Harry Potter met Hermione Granger.

==> Yesterday, **Harry Potter** met **Hermione Granger**.

x and y are both sequences, or only x is a sequence, or only y is a sequence.

Sequence Model Notation

Name Entity Recognition application is to find people's names, companies names, locations, countries names, currency names, etc. in text.

Given an input sequence of words (X), the sequence model has to automatically tell where are the proper names in this sentence.

x: Harry Potter and Hermione Granger invented a new spell.

$$x^{<1>} \quad x^{<2>} \quad x^{<3>} \quad \dots \quad x^{<9>}$$

For this example the sequence length is 9 words (features) => **$T_x=9$**

Target output y is binary vector with same length as the input
(1 if the word is name; 0 if not)

$$\mathbf{y} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{matrix} \mathbf{y}^{<1>} & \mathbf{y}^{<2>} & \mathbf{y}^{<3>} & \dots\dots & \mathbf{y}^{<t>} & & & & \mathbf{y}^{<Ty>} \end{matrix}$$

Ty=9

In this problem every input $\mathbf{x}^{(i)}$ has an output $y^{(i)}$ $\Rightarrow$ **length $\mathbf{T}\mathbf{y}=\mathbf{T}\mathbf{x}$**

Each sentence (example) can have different sequence length.

NLP - representing individual words

Natural Language Processing (NLP). Create vocabulary or download existing dictionary. For modern NLP, 10000 words is a small dictionary, 30-50 thousand is more common. Large internet companies use 1 million words dictionary.

Each word is represented by a binary vector with # elements = dimension of dictionary where only the index of the word in the dictionary = 1, all others are 0 (one-hot vector).

word index (in dictionary)

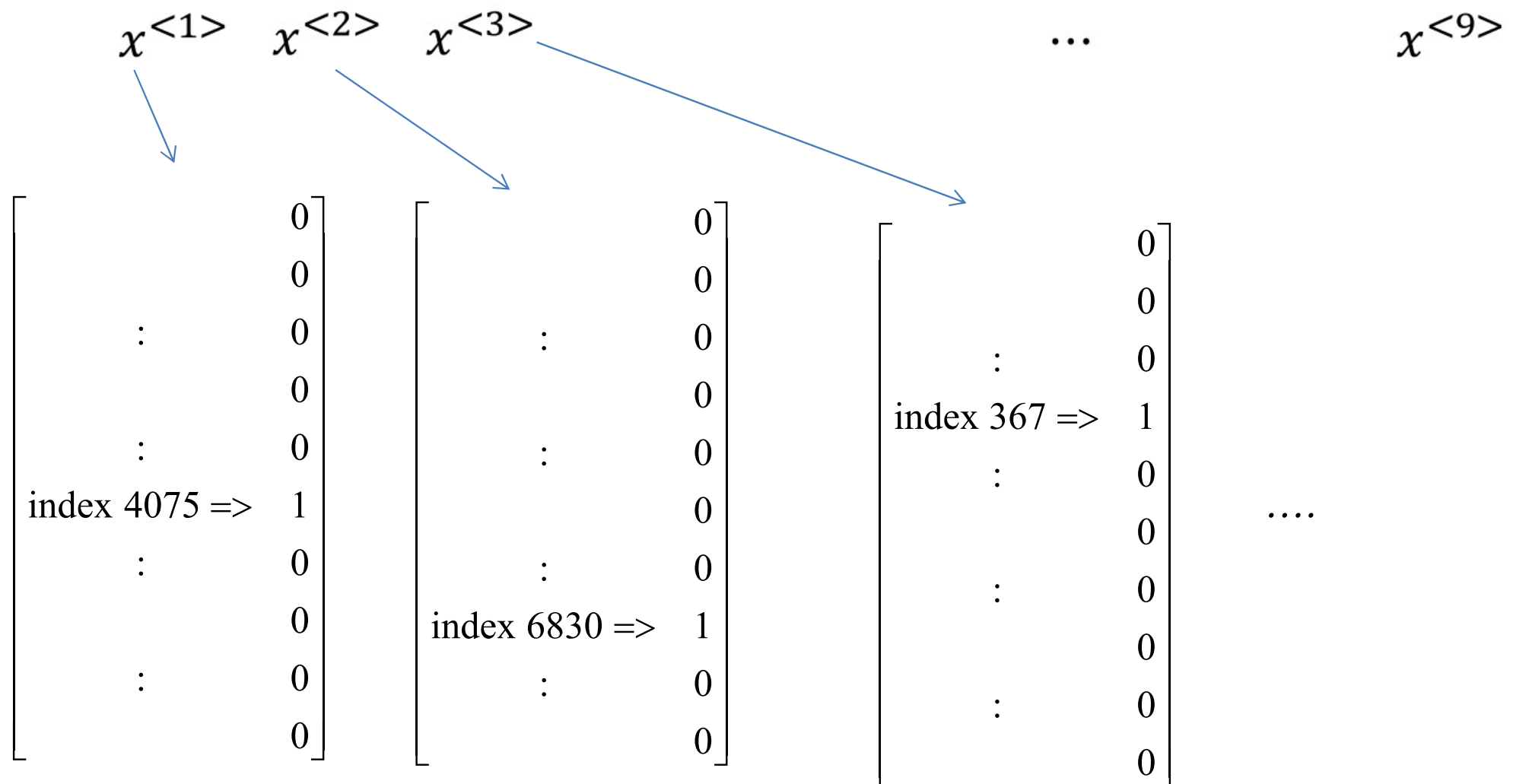
one-hot encoding

$a \Rightarrow$	1
$aaron \Rightarrow$	2
:	
$and \Rightarrow$	367
:	
$Harry \Rightarrow$	4075
:	
$Potter \Rightarrow$	6830
:	
$zulu \Rightarrow$	10000

$$Harry = \begin{bmatrix} 0 \\ 0 \\ : \\ 0 \\ : \\ 1 \\ : \\ 0 \\ : \\ 0 \end{bmatrix}$$

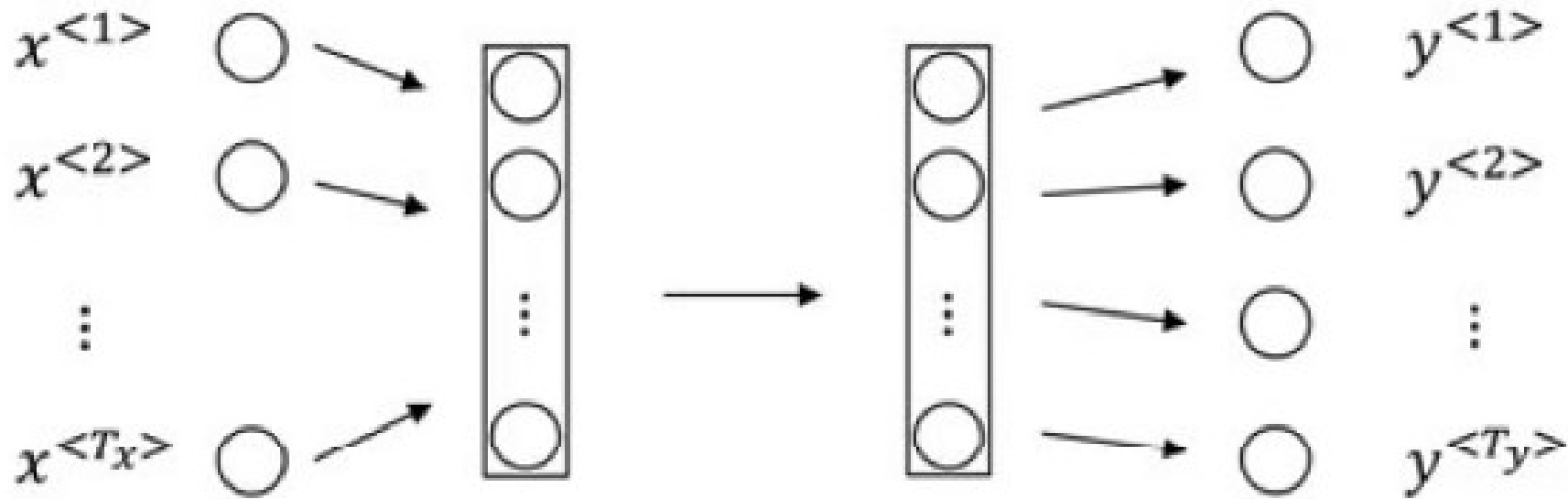
NLP – one hot encoding

x: Harry Potter and Hermione Granger invented a new spell.



<unk> - notation for words not in the dictionary. It can be added to encode all missing words that may appear in sentences.

Why not a standard neural network?



Problems:

- 1) Inputs and outputs can have different lengths in different examples.
- 2) High input dimension (e.g. T_x * dimension of dictionary) $\Rightarrow$ too many trainable parameters.

Recurrent Neural Networks (RNN) – better solution

Recurrent Neural Networks (RNN)

Read the sentence word by word (one time step per word).

Activation produced by 1st word is taken into account when read 2nd word.

Notation: inputs - $x^{<t>}$; outputs - $y^{<t>}$; hidden states - $a^{<t>}$

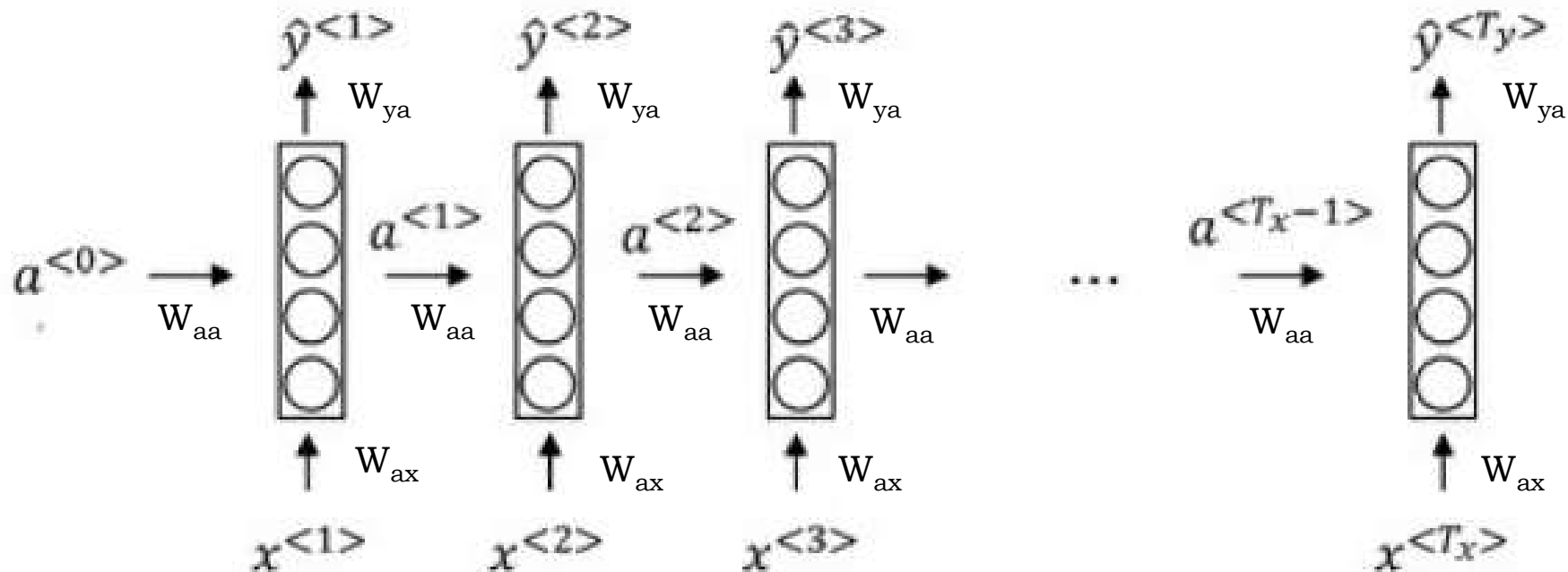
$a^{<0>}$ - initial state at time step 0, usually vector of zeros.

Previous results are passed as inputs => we get context !

A weakness of this type of uni-directional RNN is that the prediction at a certain time uses inf. that is earlier in the sequence but not latter.

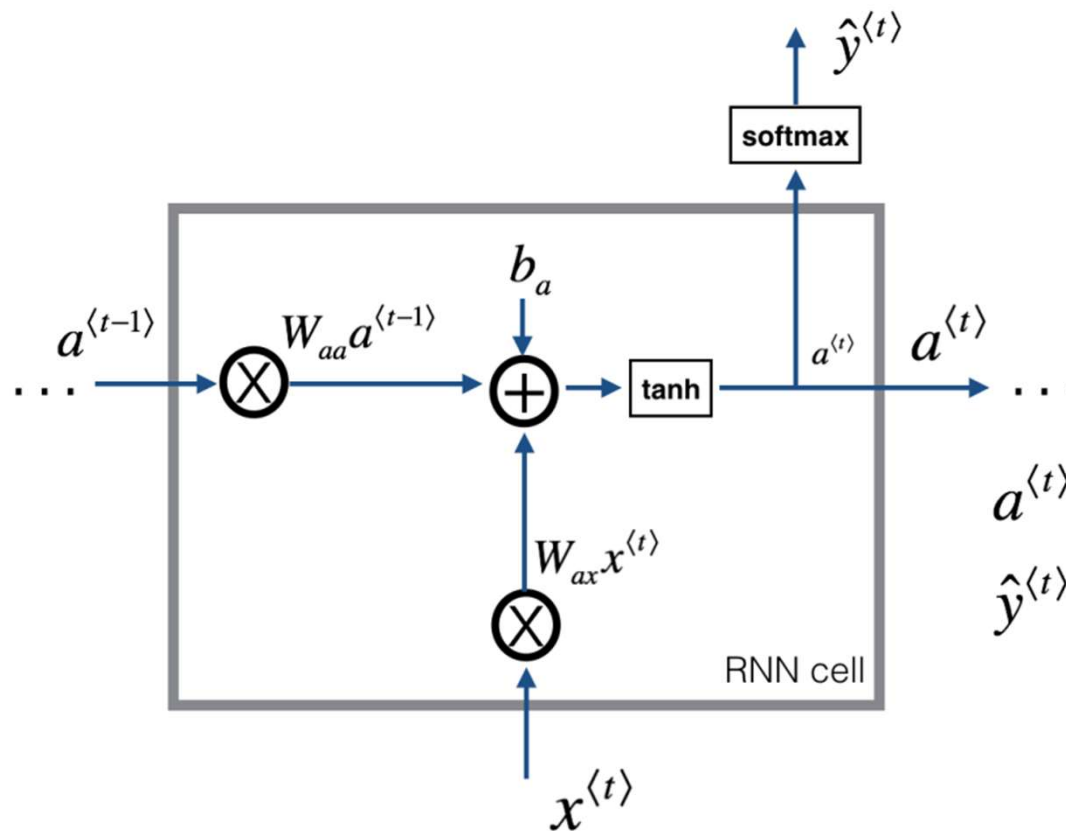
The Bidirectional RNN (BRNN) overcome this problem.

Flow chart (unrolled/unfolded RNN diagram representation) :



Basic RNN unit

Takes $x^{(t)}$ (current input) and $a^{(t-1)}$ (activation from previous step) as inputs, and computes $a^{(t)}$ (current activation). $a^{(t)}$ is then used to predict $y^{(t)}$ (current output) and passed forward to the next RNN unit (next time step). W_{aa} , W_{ax} , W_{ya} , b_a , b_y – the same RNN weights used in all time steps.



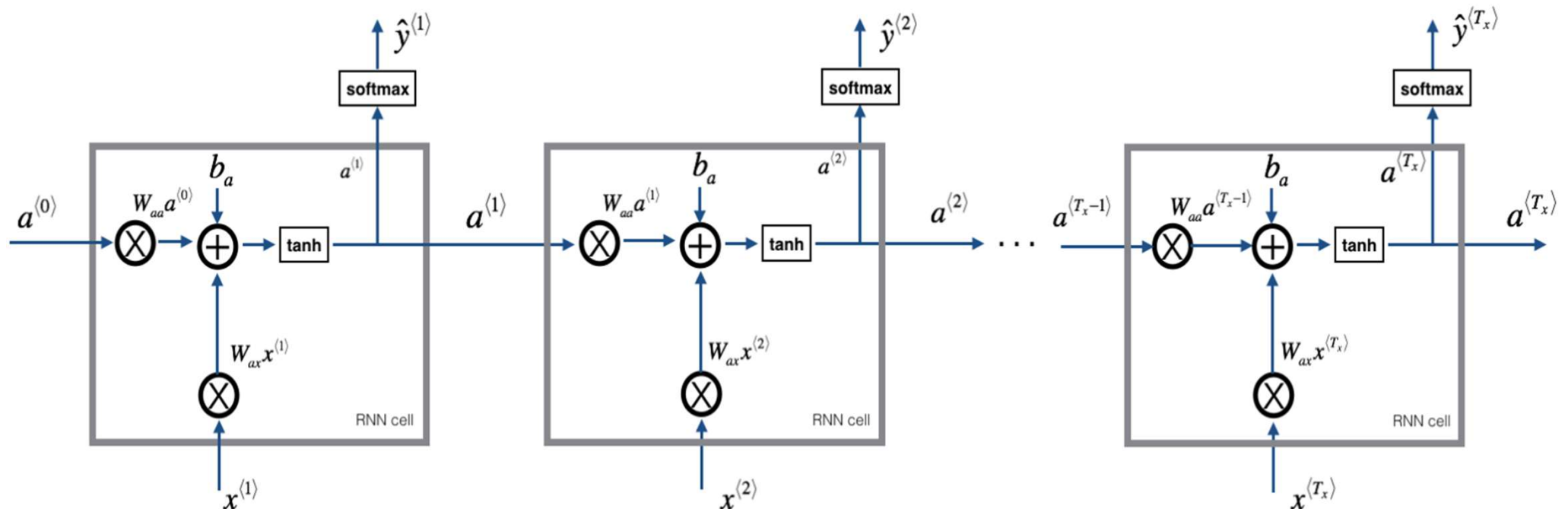
$$a^{(t)} = \tanh(W_{ax}x^{(t)} + W_{aa}a^{(t-1)} + b_a)$$
$$\hat{y}^{(t)} = \text{softmax}(W_{ya}a^{(t)} + b_y)$$

RNN forward pass

RNN is a sequence of basic RNN cells.

If input sequence is $\mathbf{x}=[\mathbf{x}^{<1>}, \mathbf{x}^{<2>}, \dots, \mathbf{x}^{<T_x>}] \Rightarrow$ RNN cell is copied T_x times.

RNN outputs $\mathbf{y}=[\mathbf{y}^{<1>}, \mathbf{y}^{<2>}, \dots, \mathbf{y}^{<T_x>}]$



Backpropagation Through Time (BPTT)

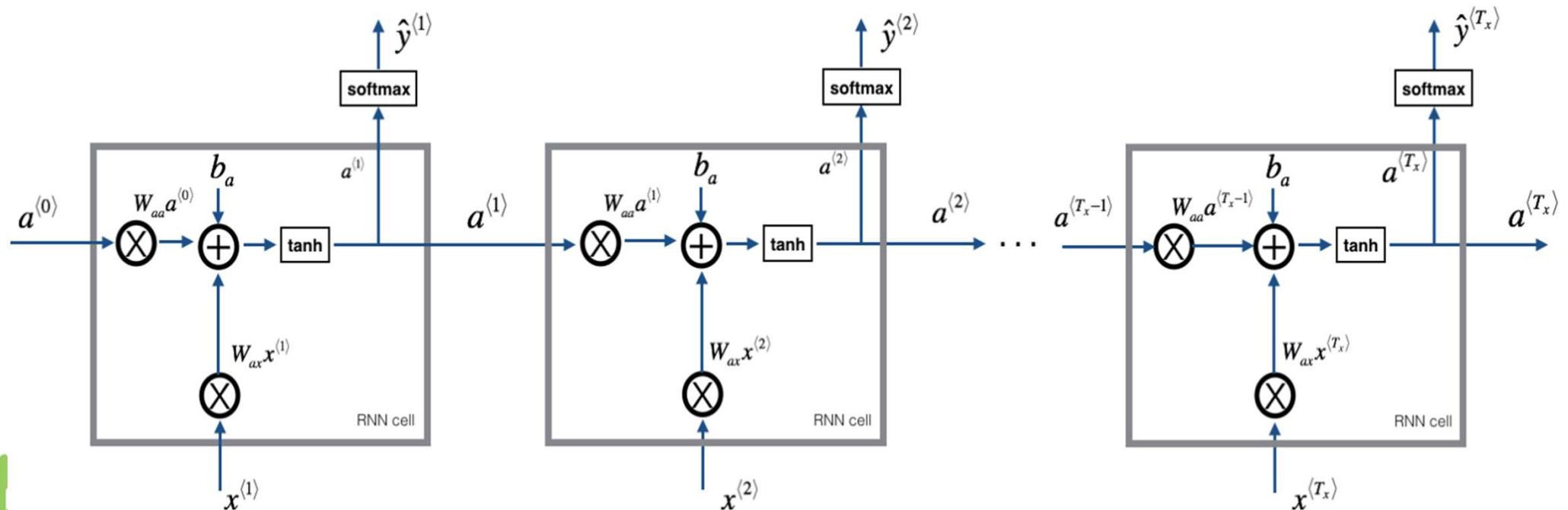
Forward propagation (for-prop): from time step 1 to time step T_x (from left to right) => compute RNN predictions.

Compute the cost (loss) function for each output $y^{(t)}$ ($J^{(t)}$)

Compute the sum of all cost (loss) functions (J_{all})

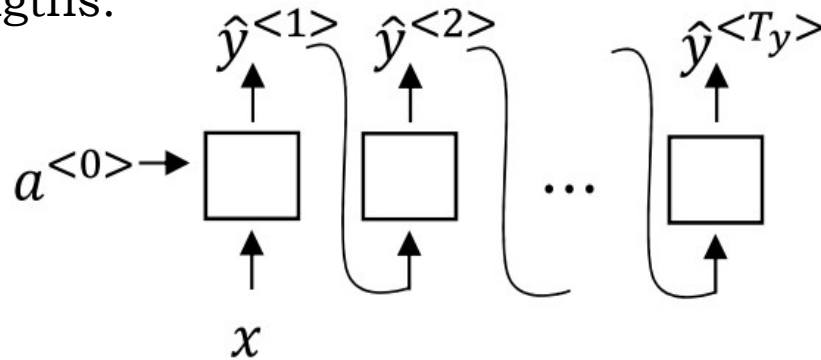
Backward propagation (back-prop) : to update RNN parameters compute the gradients of J_{all} , starting from the last time step $y^{(T_x)}$ and going back through the previous time steps down to the first time step $y^{(1)}$ (from right to left) => This is called *Backpropagation Through Time (BPTT)*.

The programming framework usually computes BPTT automatically.

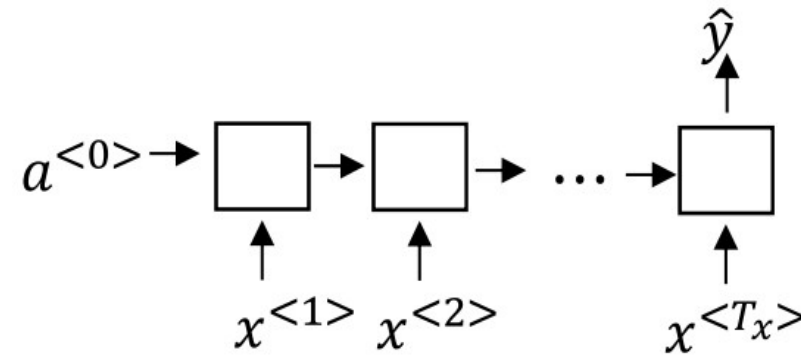


Different Types of RNN

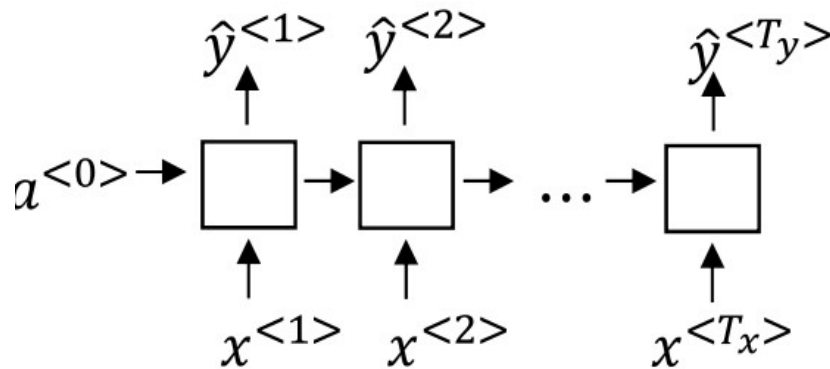
The input X and Y can be of many types and they do not have to be of the same lengths.



One to many (e.g. Music generation)

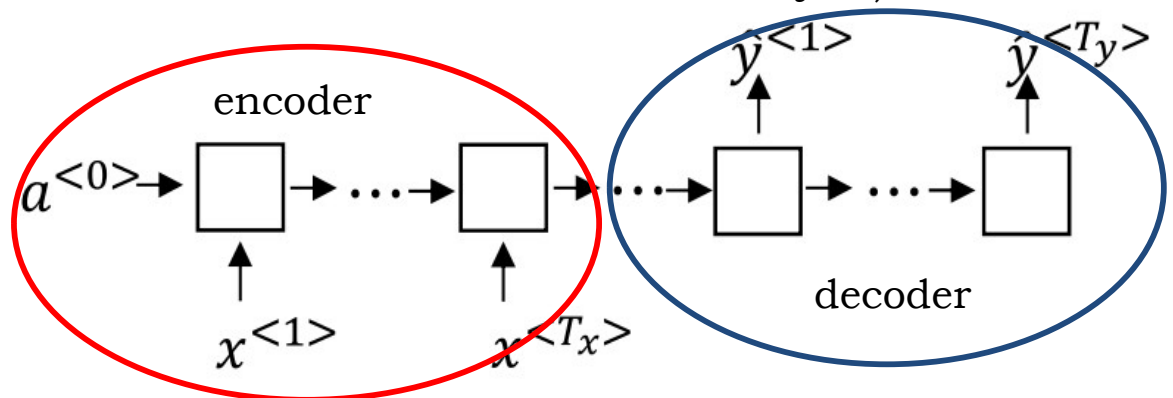


Many to one (e.g. Sentiment analysis)



Many to many

$T_x = T_y$ (e.g. Name entity rec.)



Many to many

T_x different from T_y (e.g. machine translation)

Note: Time series forecasting (many to one, or many to many)

RNN – vanishing/exploding gradients

RNN works well when each output $y^{(t)}$ can be estimated using "local" context (i.e. inf from inputs $x^{(t')}$ where t' is not too far from t).

Why ? – because RNN's suffers from vanishing gradient (similar to Deep NN).

Vanishing gradient - gradient values become very small. Layers that get small gradients stops learning. Those are usually the earlier layers in the sequence model. Because these layers don't learn, RNN's can forget what it seen in longer sequences, thus having a short-term memory.

Standard RNN is not very good in capturing long-range dependencies, ex.:

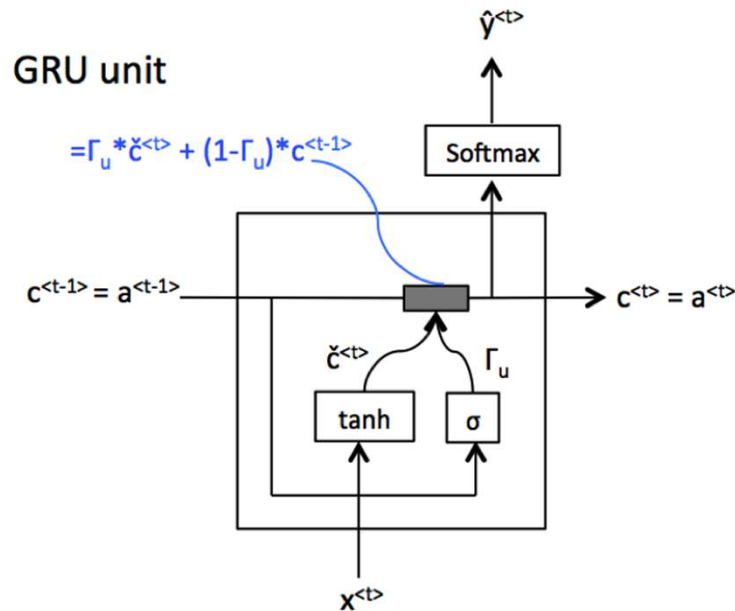
*The **girl** that entered the coffee shop with many friends **was** happy.*

*The **girls** that entered the coffee shop with many friends **were** happy.*

Need to remember “**girl-was**” or “**girls-were**”.

Long Short-Term Memory * (LSTM) and **Gated Recurrent Units** (GRU) –
Solution for vanishing gradients.

Gated Recurrent Unit (basic architecture)



c - memory cell

$c^{<t>} = a^{<t>}$ (for LSTM they are different)

GRU outputs activation value = memory cell

$\tilde{c}^{<t>} = \tanh(w_c [c^{<t-1>}, x^{<t>}] + b_c)$ - candidate

$\Gamma_u = \sigma(w_u [c^{<t-1>}, x^{<t>}] + b_u)$ - update gate

$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$ - update memory cell

$\Gamma_u \Rightarrow$ values between (0,1)

if $\Gamma_u \Rightarrow 0$, don't update $c^{<t>}$, if $\Gamma_u \Rightarrow 1$, update $c^{<t>}$.

$\Gamma_u=1$ $\Gamma_u=0$ $\Gamma_u=0$ $\Gamma_u=0$ $\Gamma_u=1$

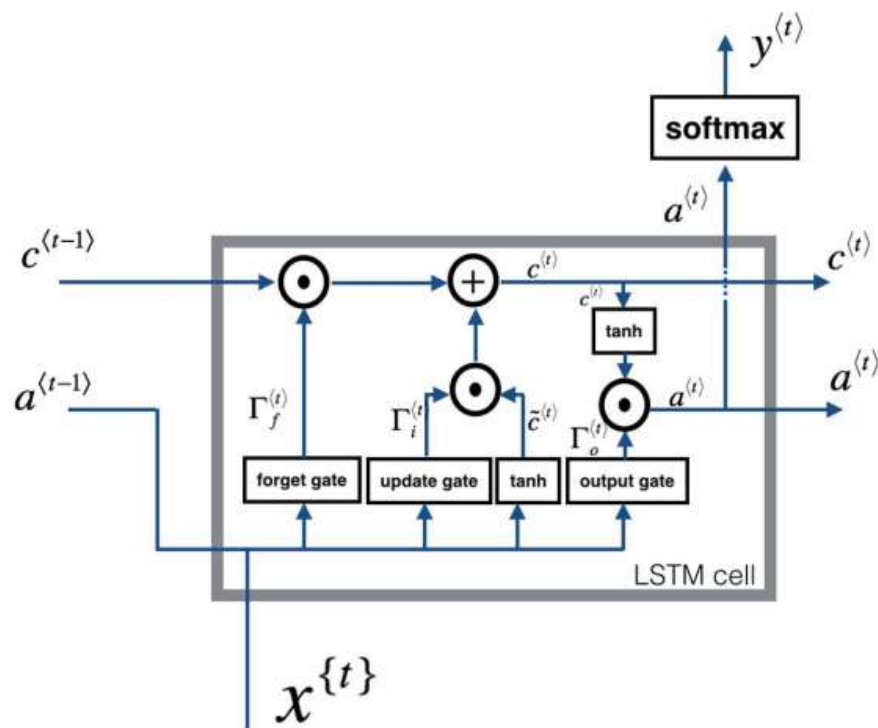
$c^{<t>} = 1$, keeps the same value of $c^{<t>}$ long time

Ex. The **girl** that entered the coffee shop with many friends **was** happy.

Memory cell **c** memorises if the girl was singular or plural even if **girl-was** are separated with many many words. Efficient against vanishing gradients.

$c^{<t>}$ and Γ_u are vectors (e.g. 100 elements/bits to update)

Long Short-Term Memory (LSTM) unit



$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c) - \text{candidate}$$

$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u) - \text{update gate}$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f) - \text{forget gate}$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o) - \text{output gate}$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>} - \text{update memory cell}$$

$$a^{<t>} = \Gamma_o * \tanh(c^{<t>}) - \text{activation}$$

LSTM outputs both activation value and memory cell

Gates (Γ_f , Γ_u , Γ_o) contain sigmoid activations $\Rightarrow$ values between 0 and 1.

Forget gate decides what is relevant to keep from prior steps.

Inf from the previous hidden state and from the current input is passed through the sigmoid function. Output values between 0 and 1.

Closer to 0 means to forget, closer to 1 means to keep.

Update gate decides what inf is relevant to add from the current step.

Output gate determines what the next hidden state should be.

GRU & LSTM

LSTM and GRU are two variations of RNNs to capture better long range dependencies (connections) in sequences.

They mitigate short-term memory using mechanisms called memory cell and gates.

Gates remember (keep saved) bits of inf for many time steps.

Ex.: Read text, and use LSTM/GRU to keep track of grammatical structures => if the subject is singular or plural. If the subject changes from a singular word to a plural word, forget the previously stored memory value of the singular state.

LSTM's and GRU's are successfully applied in speech recognition, speech synthesis, natural language understanding, time series forecasting, etc.

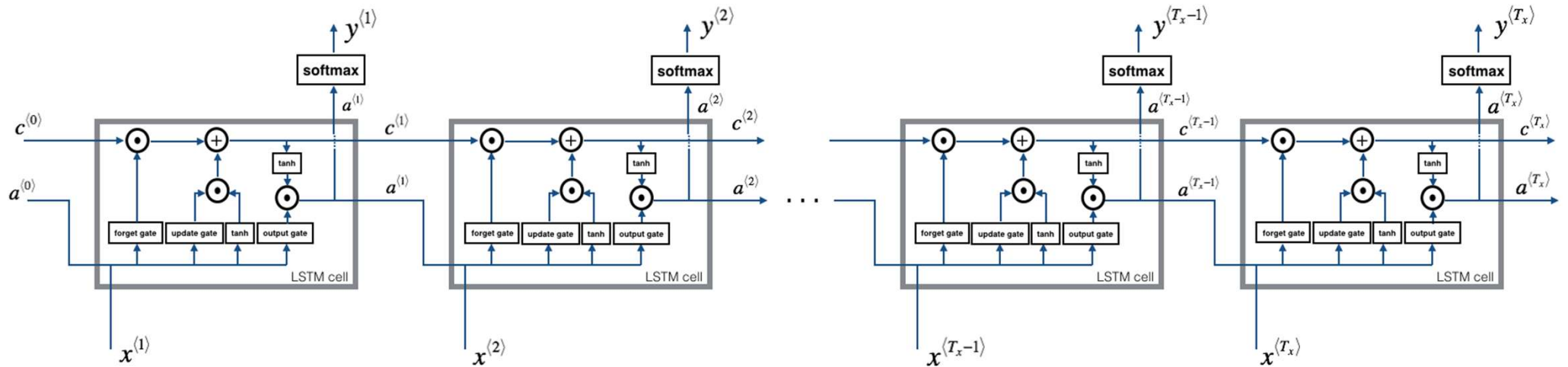
When to use LSTM or GRU ? - there is not a consensus.

LSTM appeared first (1997), GRU (2014) is a simplification of LSTM.

LSTM is more powerful (3 gates in LSTM, 1 gate in GPU).



LSTM network



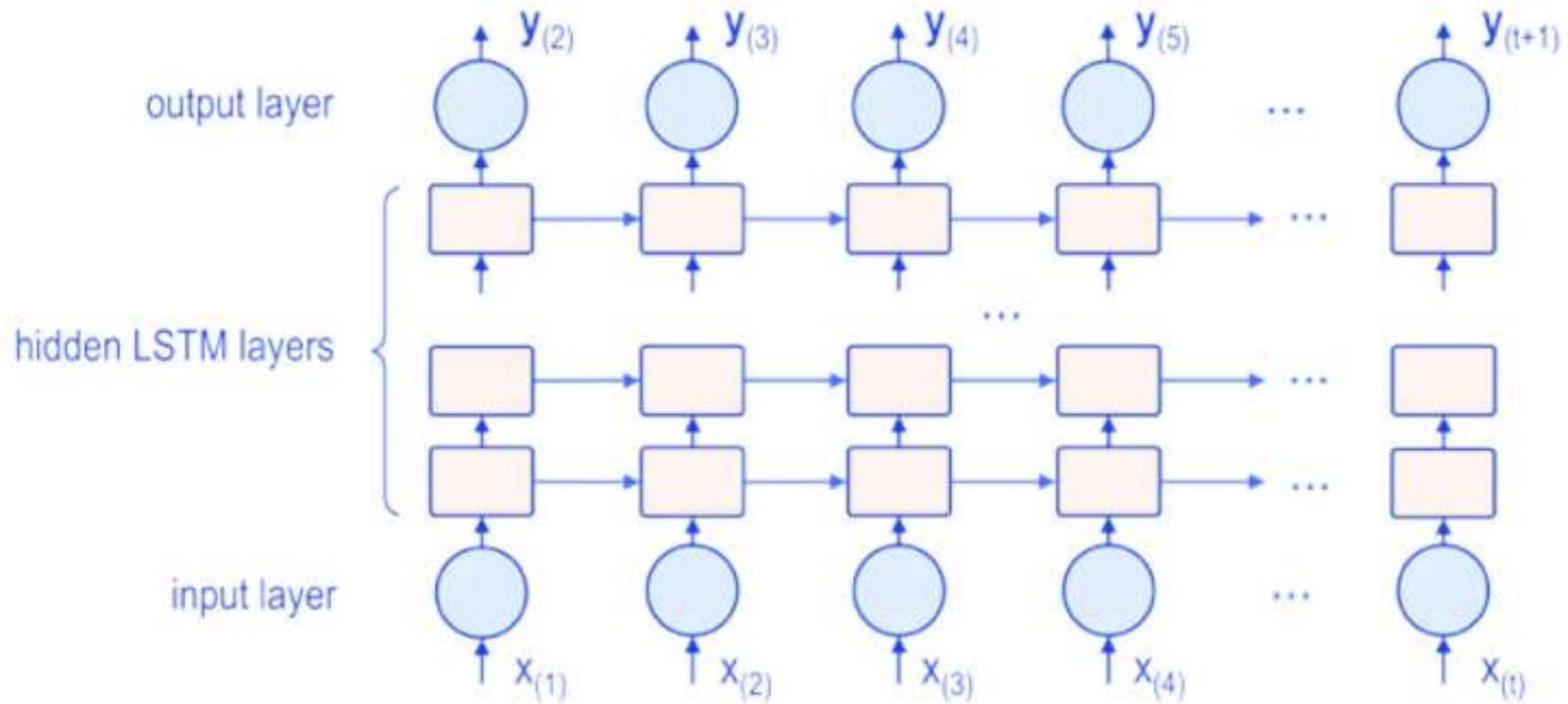
LSTM network is a temporal sequence of LSTM units.

There is a line at the top that shows how LSTM can memorise and pass certain values $c^{<t>}$ through several temporal steps.

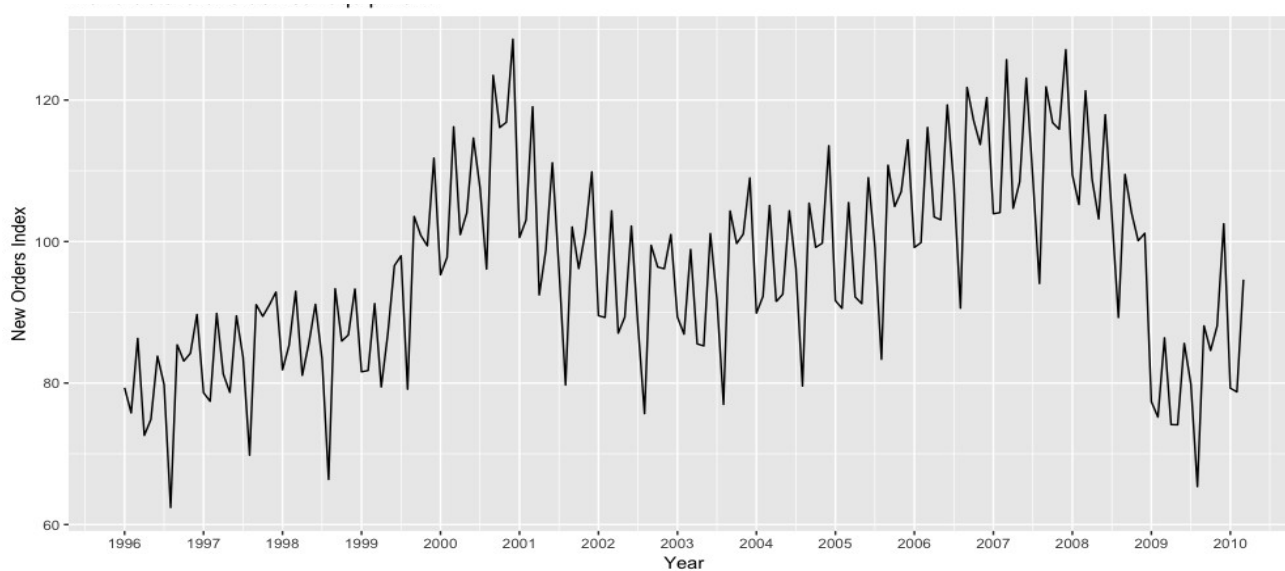
Other versions : the gate's values depend also on $c^{<t-1>}$. (peephole connection)

The gates are vectors (e.g. 100 elements).

Deep LSTM



Time Series Data

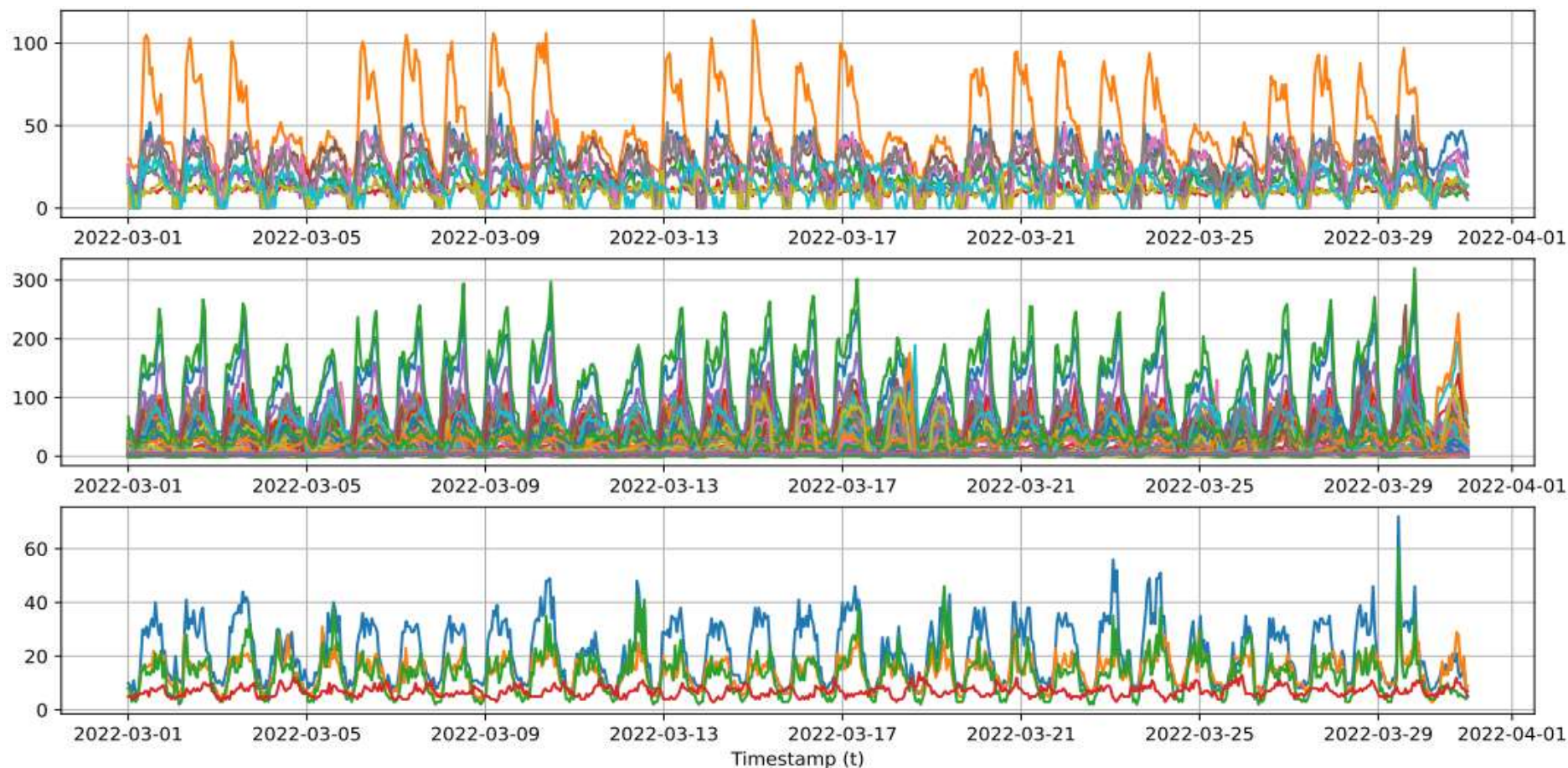


Time Series (TS) - collection of samples recorded at a sequence of time intervals

TS forecasting (prediction) => based on past samples, predict future trends, seasonality, anomalies, etc. Many applications:

- Key Performance Indicators (KPIs) in networks: e.g. traffic prediction
- Smart Homes – indoor temperature prediction
- Weather forecasting
- Financial data forecasting – e.g. bitcoin price

Time Series Properties

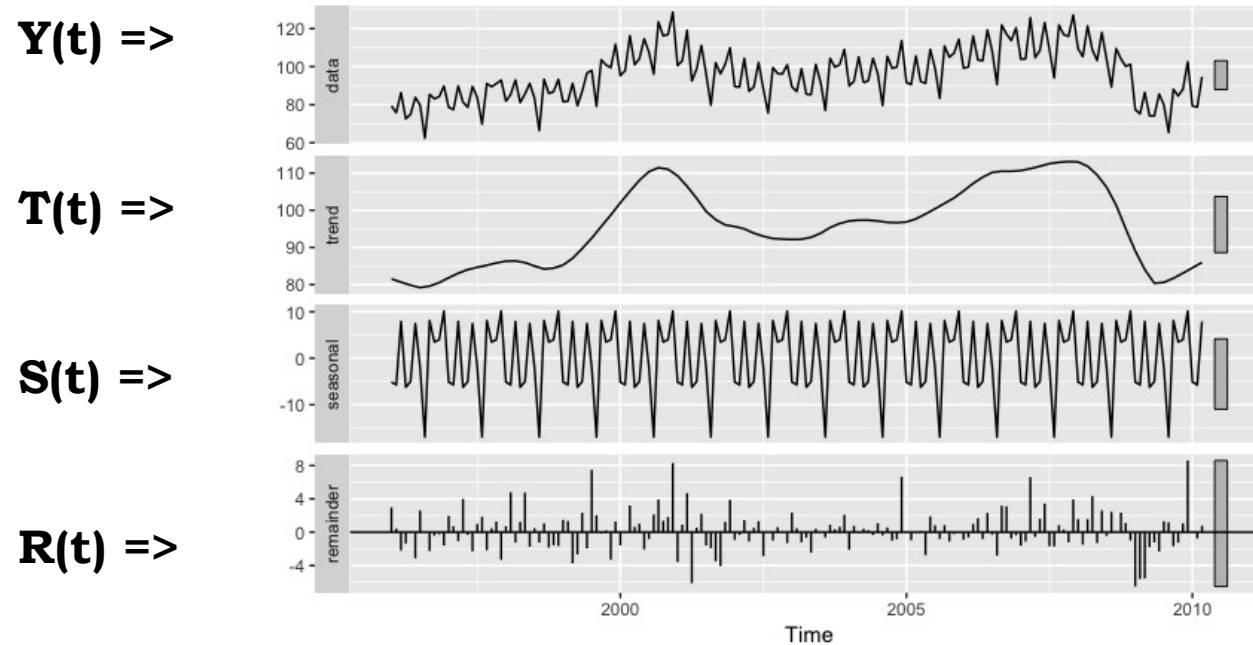


Stationarity - TS is stationary when the mean and variance remain constant over time.

Seasonality – TS variations at specific time-frames (i.e. people buy more Christmas trees during Christmas, people eat more ice-cream in summer).

Auto-correlation - correlation between the current value and the value from previous times (lags).

Time Series Decomposition



If TS $Y(t)$ shows some seasonality (e.g. daily, weekly, yearly) it can be decomposed into 3 components:

$$Y(t) = S(t) + T(t) + R(t)$$

$T(t)$ - trend, estimated through the mean of last n samples (rolling mean)

$S(t)$ - seasonal component

$R(t)$ - remaining component

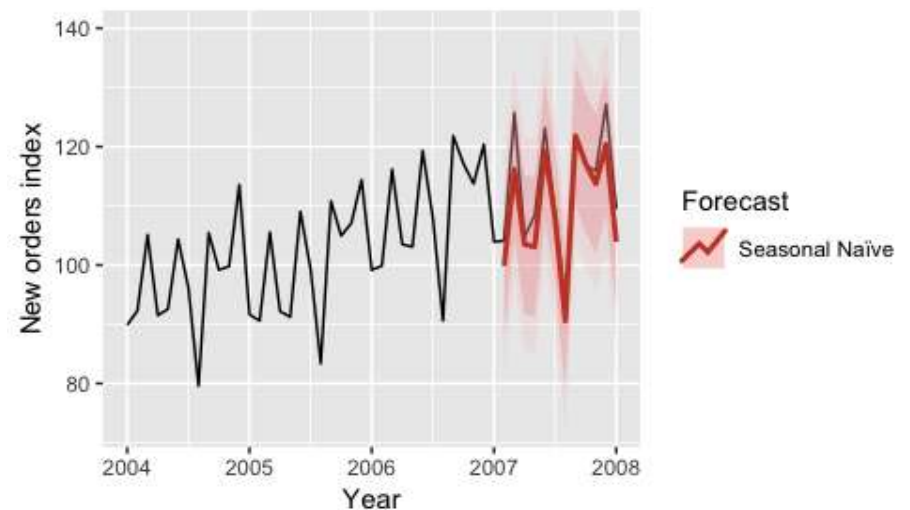
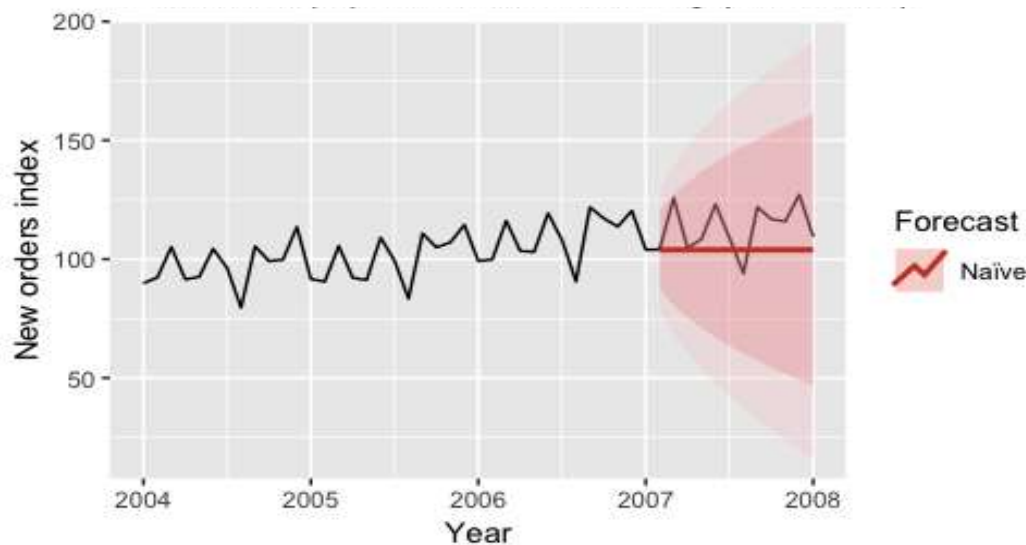
Time Series Forecasting – classical methods

1) Naïve model : the forecasts for every horizon (h) = last observed value:

$$\hat{Y}(t+h | t) = Y(t)$$

2) SNaïve (Seasonal Naïve) model: TS has a seasonal component with period of seasonality T :

$$\hat{Y}(t+h | t) = Y(t+h-T)$$

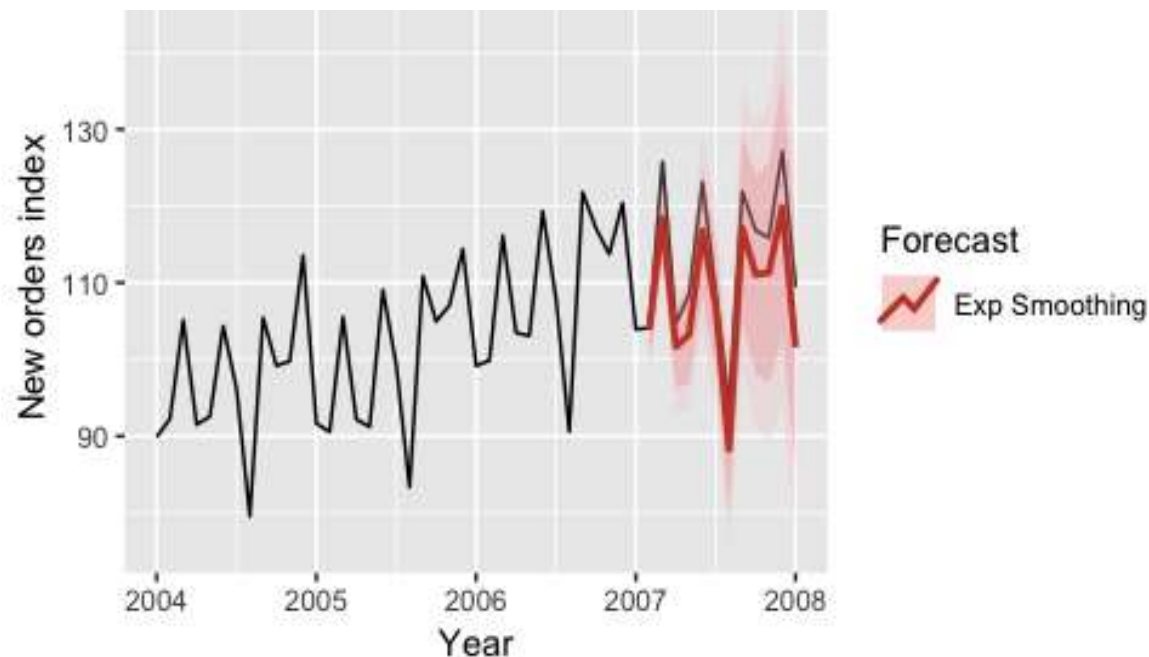


Time Series Forecasting – classical methods

3) Exponential smoothing

Weighted average of past observations, weights decrease exponentially as go back in time.

$$\hat{Y}(t+h | t) = \alpha Y(t) + \alpha(1-\alpha)Y(t-1) + \alpha(1-\alpha)^2 Y(t-2) + \dots, 0 < \alpha < 1 \text{ (basic model)}$$



Time Series Forecasting – classical methods

4) **ARIMA** - Auto-Regressive (Integrated) Moving Average

ARMA (stationary signals), ARIMA (both stationary & nonstationary)

Auto-Regressive model - linear combination of past values of TS.

Moving-Average model – lin. combination of past forecasting errors.

Integrated (use differences) : $\mathbf{Y(t) = Y(t) - Y(t-1)}$

ARIMA = AR + I + MA components

$$\hat{Y}(t+h | t) = a_1 Y(t) + a_2 Y(t-1) + \dots + a_p Y(t-p) + b_1 E(t) + b_2 E(t-1) + \dots + b_q E(t-q)$$

5) SARIMA model (Seasonal ARIMA) extends ARIMA by adding a linear combination of seasonal past values and forecast errors.

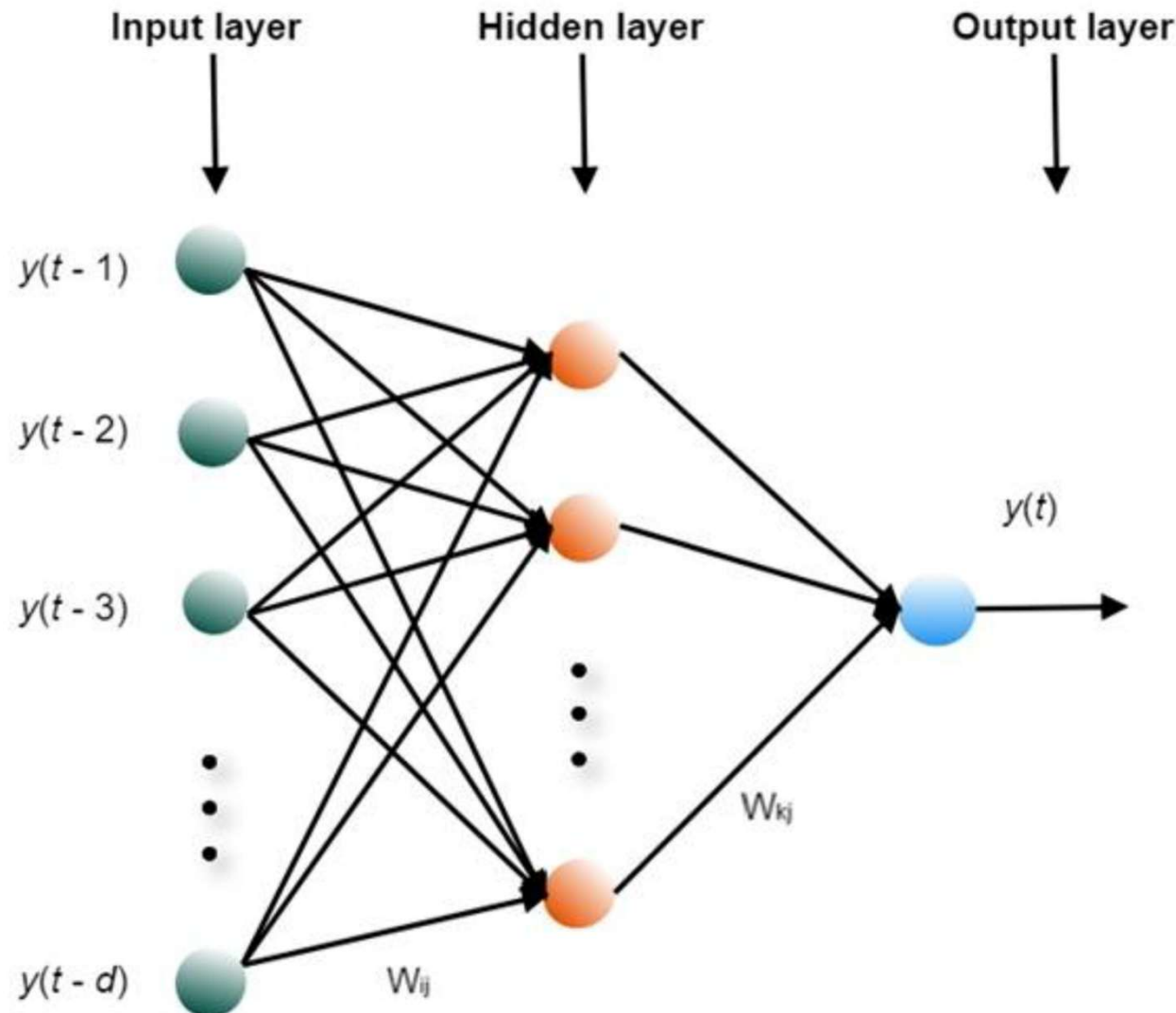
ARIMA (p,d,q)(P,D,Q)

p=> autoregressive lags; q=> moving average lags;

d=> difference in the order

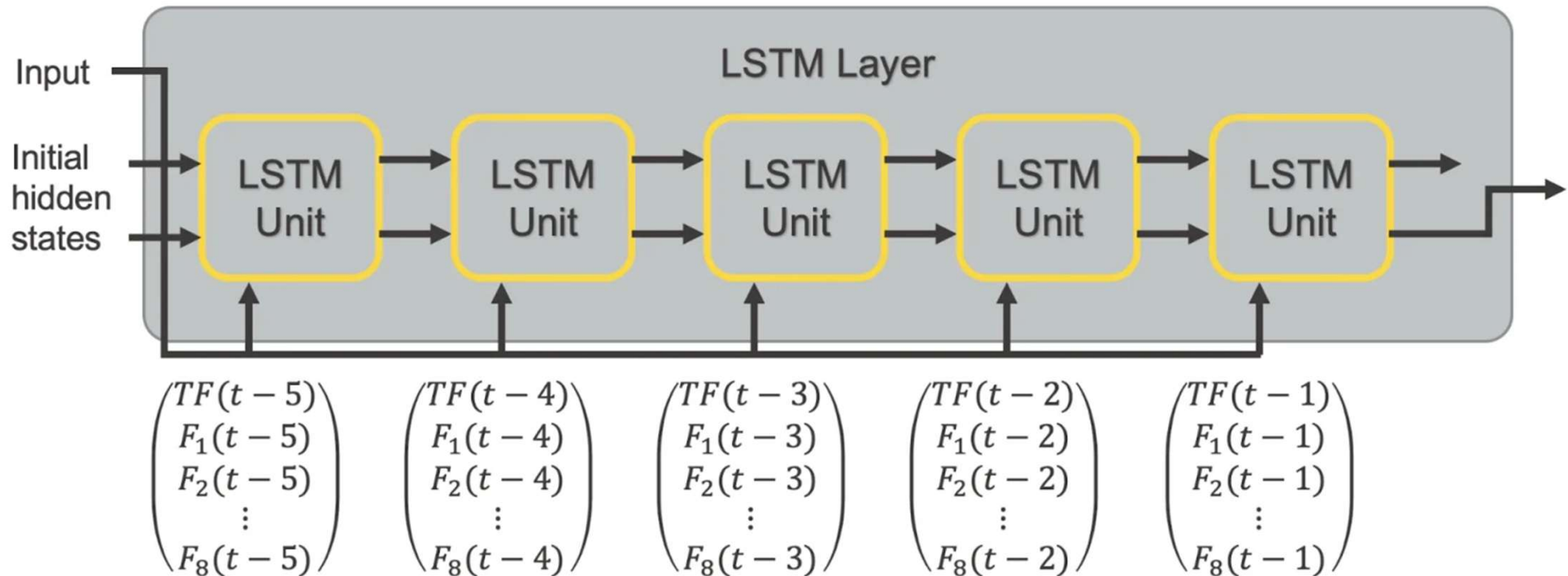
Time Series Forecasting – ML approach

Non-linear Auto Regressive Neural Network (NARN)
Classical fully connected shallow (3 layers) ANN
d- lag hyperparameter (time lag, model memory)



Time Series Forecasting – DL approach

Long Short Term Memory (LSTM) architecture or GRU (Gated Recurrent Unit) architecture



Multivariate prediction model

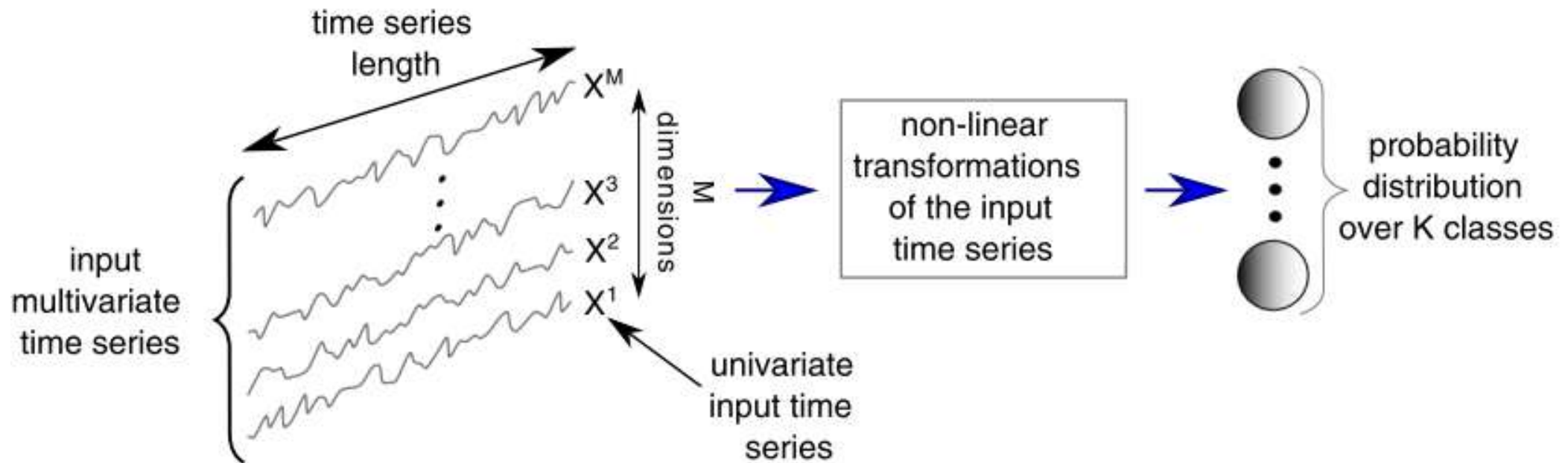
Predict $TF(t)$ – the output

$TF, F_1, F_2, \dots, F_8$ – predictors/features

ex. lag $d=5$

Multivariate TS classification

Conv NN



Recommended further reading

- Understanding LSTM Networks – colah’s blog :

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

- The Unreasonable Effectiveness of Recurrent Neural Networks - Andrej Karpathy blog:

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>